Algorithms — Unified Notes from Skiena (TADM 3e) + CLRS (4e)

Audience: working backend engineer targeting SDE-2 / Senior SWE interviews and competitive programming.
Assumed background: solid programming, basic data structures, basic C++.
All implementation code is C++17.

C++ conventions used everywhere in these notes

Every ```` ```cpp ```` block in every module compiles under `g++ -std=c++17 -Wall -Wextra` and assumes exactly this prelude:

```cpp
#include <bits/stdc++.h>    // one header, the whole standard library (GCC/Clang only)
using namespace std;        // no std:: prefixes anywhere in these notes
```

- **`using namespace std;` is assumed, so `std::` never appears.** In production code you would not do this; in notes, in competitive programming, and on a whiteboard, the `std::` noise buys nothing and costs readability. The one place it bites is **name lookup** — a class member or local name silently hides the `std` version (see the `mergeRuns` comment in M09). Where that can happen, the code says so.
- **`#include <bits/stdc++.h>` is a GCC/Clang convenience header**, not standard C++. Real projects list explicit headers (`<vector>`, `<algorithm>`, `<queue>`, …); judges and interviews accept the shortcut.
- **Every module ends with two C++ sections:**
    - **C++ Toolkit for This Module** — the language features that module's code leans on, explained from Weiss, *Data Structures and Algorithm Analysis in C++* (4th ed.), ch. 1.
    - **Appendix — C++ for Every Pseudocode Block** — a heavily-commented, runnable translation of *every* pseudocode block in that module. Each pseudocode block in the body carries a link straight to its

implementation.

- **Body code vs appendix code.** Body implementations are the *practical* version — the one you would actually write. Appendix implementations are *literal translations of the pseudocode*, line for line, so you can see the correspondence. They are deliberately different, and comparing them is the point.
- **Third book used only for the C++ teaching**, not for algorithms: Mark Allen Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Cited as `[Weiss §1.5.3, p.25]`.
- **Every module also ends with a `Practice` section** — specific problems by number and title on LeetCode, plus CSES and Codeforces tag pages.

Verified state of the code and links: 30 translation units (a body and an appendix per module) all compile under `g++ -std=c++17 -Wall -Wextra`; 435 internal links resolve with 0 broken; 95 `→ C++ implementation:` links cover all 71 procedure-pseudocode blocks plus 24 algorithmic recurrences and formulas.

How these notes are built

Two books, one set of notes. Every topic is written once, as a **Unified Understanding**, drawing from
whichever book explains it best:

SOURCE	WHAT IT CONTRIBUTES
Skiena — The Algorithm Design Manual, 3e	Intuition, problem modeling, pattern recognition, "which algorithm do I actually reach for", war stories, implementation pragmatics, counterexample hunting
CLRS — Introduction to Algorithms, 4e	Formal definitions, loop invariants, recurrence solving, proofs, rigorous complexity, theoretical depth

Where the books genuinely diverge in emphasis, that is called out explicitly as **Skiena emphasis** / **CLRS emphasis**. Anything not supported by either book is quarantined
under `###` **Outside / Engineering Context**.

Page references look like `[CLRS §2.1, p.17]` and `[Skiena §1.3, p.11]` so you can go back to the source fast.

Module map

Status: ✅ written · ⏳ planned (no file yet — links in the modules point back here).

Part I — Foundations

#	MODULE	PRIMARY SOURCES	STATUS
M01	Foundations of Algorithm Design	CLRS 1–2 · Skiena 1	✅
M02	Asymptotics & the Analysis Toolkit	CLRS 3 + App. A · Skiena 2	✅
M03	Divide & Conquer and Recurrences	CLRS 4 · Skiena 5	✅
M04	Randomization & Probabilistic Analysis	CLRS 5 · Skiena 6	✅

Part II — Sorting and Data Structures

#	MODULE	PRIMARY SOURCES	STATUS
M05	Sorting & Order Statistics	CLRS 6–9 · Skiena 4	✅
M06	Elementary Data Structures	CLRS 10 · Skiena 3.1–3.3, 3.5, 3.8	✅
M07	Hashing	CLRS 11 · Skiena 3.7, 6	✅
M08	Search Trees & Augmentation	CLRS 12, 13, 17, 18 · Skiena 3.4	✅
M09	Amortized Analysis	CLRS 16	✅
M10	Disjoint Sets / Union-Find	CLRS 19 · Skiena 8.1.3	✅

Part III — Design Techniques

#	MODULE	PRIMARY SOURCES	STATUS
M11	Dynamic Programming	CLRS 14 · Skiena 10	✅
M12	Greedy Algorithms	CLRS 15 · Skiena (throughout)	✅

Part IV — Graphs

#	MODULE	PRIMARY SOURCES	STATUS
M13	Graph Representation & Traversal	CLRS 20 · Skiena 7	✅
M14	Minimum Spanning Trees	CLRS 21 · Skiena 8.1	✅
M15	Shortest Paths	CLRS 22–23 · Skiena 8.3–8.4	✅
M16	Network Flow & Matching	CLRS 24–25 · Skiena 8.5	⏳

Part V — Search, Strings, Intractability

#	MODULE	PRIMARY SOURCES	STATUS
M17	Combinatorial Search & Backtracking	Skiena 9	⏳
M18	String Matching & Suffix Structures	CLRS 32 · Skiena 3.9, 21	⏳
M19	NP-Completeness & Reductions	CLRS 34 · Skiena 11	⏳
M20	Coping With Hard Problems	CLRS 35 · Skiena 12	⏳

Part VI — Specialized Topics

#	MODULE	PRIMARY SOURCES	STATUS
M21	Number-Theoretic Algorithms	CLRS 31	⏳
M22	Linear Programming	CLRS 29 · Skiena 13.6	⏳
M23	Matrix Operations, Polynomials & FFT	CLRS 28, 30	⏳
M24	Parallel & Online Algorithms	CLRS 26–27	⏳
M25	Machine-Learning Algorithms	CLRS 33	⏳
M26	Geometry & the Algorithm Catalog	Skiena 13, Part II	⏳

Part VII — Revision

#	MODULE		STATUS
M27	Master Cheat Sheet & Recognition Playbook	cross-module	⏳

CLRS ↔ Skiena crosswalk

Use this when you want to read the *books* rather than the notes, and want the matching chapter in the other book.

TOPIC	CLRS 4E	SKIENA 3E
What is an algorithm; correctness	Ch. 1, §2.1	Ch. 1
Counterexamples, heuristics vs algorithms	— (implicit)	§1.1–1.3
Modeling with combinatorial objects	—	§1.5
RAM model	§2.2	§2.1
Loop invariants	§2.1	§1.4 (as induction)
Asymptotic notation	Ch. 3	§2.2–2.4
Summations, logarithms	App. A	§2.6–2.8
Divide & conquer, recurrences	Ch. 4	Ch. 5
Master theorem	§4.5	§5.5
Randomized analysis, indicator RVs	Ch. 5	§4.6.2, Ch. 6
Heaps / priority queues	§6	§3.5, §4.3
Quicksort	Ch. 7	§4.6
Linear-time sorting, lower bound	Ch. 8	§4.7
Selection / medians	Ch. 9	§17.3 (catalog)
Arrays, lists, stacks, queues	Ch. 10	§3.1–3.2
Hashing	Ch. 11	§3.7, Ch. 6
BSTs	Ch. 12	§3.4
Balanced trees	Ch. 13 (RB)	§3.4.3 (survey)
Augmenting structures	Ch. 17	§3.8
B-trees	Ch. 18	§3.4.3 (mention)
Dynamic programming	Ch. 14	Ch. 10
Greedy	Ch. 15	§1.2, Ch. 8 (MST)
Amortized analysis	Ch. 16	§3.1.1 (dynamic arrays)
Union-Find	Ch. 19	§8.1.3
BFS/DFS, topological sort, SCC	Ch. 20	Ch. 7
MST	Ch. 21	§8.1–8.2
Shortest paths	Ch. 22–23	§8.3–8.4
Network flow	Ch. 24–25	§8.5
Backtracking / branch & bound	—	Ch. 9
String matching	Ch. 32	§3.9, Ch. 21
NP-completeness	Ch. 34	Ch. 11

Approximation / heuristics	Ch. 35	Ch. 12
Number theory, RSA	Ch. 31	§16.x (catalog)
Linear programming	Ch. 29	§13.6
FFT, matrix ops	Ch. 28, 30	§16.x (catalog)
Parallel, online	Ch. 26–27	—
Machine learning	Ch. 33	—
Computational geometry	—	Ch. 20 (catalog)
"How to design algorithms"	—	Ch. 13

Notable coverage gaps to be aware of:

- **Only in CLRS:** amortized analysis as a formal technique, red-black trees in full, B-trees, van-Emde-Boas-style augmentation, matrix operations, linear programming internals, FFT, number theory / RSA, parallel algorithms, online algorithms, ML algorithms, the formal Cook–Levin machinery.
- **Only in Skiena:** counterexample-hunting methodology, problem modeling, backtracking as a first-class technique, simulated annealing and local search, the war stories, the 75-problem catalog, and the "how to design algorithms" decision procedure.

Suggested study order

If you are preparing for interviews (12–14 weeks):

1. M02 → M03 (analysis machinery — everything else depends on it)
2. M05 → M06 → M07 → M08 → M09 → M10 (data structures, the interview bread and butter)
3. M11 → M12 (DP and greedy — the highest-yield technique modules)
4. M13 → M14 → M15 (graphs)
5. M18 (strings), M17 (backtracking)
6. M16 (flow), M19 (NP-completeness) — appear in senior interviews as "is this even tractable?"
7. M27 (cheat sheet) — revise weekly from here once modules are done
8. M01, M04, M20–M26 as depth/breadth passes

If you are studying for mastery: front to back.

How to revise

Each module ends with three revision aids:

- **Chapter-in-one-page** — the compressed version.
- **Recognition table** — problem clue → technique.
- **Self-test questions** — if you can answer these without the notes, the module is done.

Do not re-read modules linearly. Re-read the recognition tables, then attempt the self-test,
then only open the section you failed.